Senior Data Scientist

Applied AI engineering exercise

This exercise reflects the kind of work carried out at Brdge day to day: taking a real problem and a mixed set of source material, deciding on the right technical approach, and building something that works and can be reasoned about. It is designed to show us how you architect an AI system, how you handle the parts that are genuinely hard, and how you write code another engineer could pick up.

> **Overview**
>
> Language: Python. Beyond that, use whatever libraries and tooling you would reach for in practice.
>
> You may use any LLM provider. Keep any API usage modest with a smaller, lower-cost model.
>
> Submission: a link to a Git repository (GitHub, GitLab, or similar).

This is broader than can be finished perfectly, and that is deliberate. We are more interested in how you prioritise and reason than in whether every part is complete. Get as far as you sensibly can, keep the core working, and use the notes section to set out what you would do with more time. What you choose to leave out is as informative as what you build.

Background

A client is building an assistant that helps people work through everyday emotional states. When a user says how they are feeling, the assistant needs to respond in a way that is genuinely helpful and, above all, safe. It should never improvise clinical advice, and it must recognise when an input needs a human rather than an automated response.

You have been given a small, synthetic knowledge pack that mirrors the structure of the real thing. It contains three deliberately different kinds of data, described below. Recognising that they call for different handling is part of the exercise.

The data pack

File	What it is
knowledge_base.json	The structured, safety-relevant core. Emotional states linked by explicit ids to frameworks, practices, and inquiry and mirror patterns. The relationships are authored and exact.
classifier_seeds.json	A labelled seed set mapping example utterances to a state, an intensity, and a safety flag (none, watch, or escalate).
Wisdom corpus (linked)	The unstructured component. Four public sources, listed below, which you fetch and process yourself. Different traditions and formats on purpose.

A short README is included in the pack describing the join keys and the safety flags. Please read it before starting.

Wisdom corpus sources

Use these four public sources as the free-text corpus for the reflective path. Fetch and process them yourself. They span different traditions and formats on purpose, an HTML page, a PDF, a full text file, and a workbook, which mirrors the kind of hybrid retrieval work we do in practice. Handling that variety well, not just retrieving from one clean format, is part of what we are looking at.

- **Tao Te Ching, translated by James Legge.** https://sacred-texts.com/tao/taote.htm
- **The Bhagavad Gita (The Song Celestial), translated by Edwin Arnold.** https://www.gutenberg.org/files/2388/2388-h/2388-h.htm
- **Self-help CBT techniques, NHS Every Mind Matters.** https://www.nhs.uk/every-mind-matters/mental-wellbeing-tips/self-help-cbt-techniques/
- **Valued Living, a self-help guide to living effectively with emotional pain (ACT), Shane Ford.** https://contextualscience.org/sites/default/files/Self-Help%20ACT%20Manual%20for%20Anxiety%20&%20Depression.pdf

The task

Build an agentic assistant that takes a user message and produces an appropriate, grounded response by deciding what to do and calling the right tools, rather than pushing every input through a single pipeline. This is the heart of the exercise: the routing and orchestration, not just retrieval.

What the assistant needs to handle

Different messages need different handling. At a minimum, your assistant should work out which of the following a given message calls for, and act accordingly.

- **Structured guidance.** When a message maps to an emotional state, look up the correct framework, practice, and pattern from the structured knowledge base and respond from that. This is an exact, keyed lookup, not a similarity search.
- **Reflective or open input.** When a message is looser or reflective, draw on the free-text corpus of public sources, where retrieving by meaning is a sensible fit. The four sources differ in tradition and format, so retrieving the right passage and being clear which source it came from is part of what we are testing.
- **Safety.** When a message signals real distress or risk, it must not be met with an ordinary exercise. It should route to a safety response that surfaces human and crisis support. This path takes priority over everything else.
- **Nothing suitable.** When there is no good match, the assistant should say so honestly rather than inventing a response.

Components we expect to see

- **A router or classifier** that reads the incoming message and decides which path and tools it needs. The seed set is your starting point.
- **A structured lookup** over the knowledge base that returns the correct linked content for a state by following the join keys.

- **A retrieval tool** over the four public sources for the reflective path, able to retrieve from whichever source or sources actually answer the message.
- **A safety gate** that intercepts escalation cases before anything else runs.
- **An API endpoint** that takes a message and returns the response, with some indication of which path was taken and what it drew on.

Robustness

The endpoint should be robust, not just functional. We are particularly interested in how you handle invalid or empty input, a message the system cannot place, and failures from the underlying model or services such as timeouts, rate limits, and errors. Sensible validation, error handling, and clear responses matter more than extra features.

Latency is also worth thinking about. Several steps, routing, lookup or retrieval, and a model call, can add up, and this is something we pay close attention to in our own projects. A brief note on where the time goes and what you would do to keep it responsive is welcome, even if you do not optimise it fully.

Architecture note

Alongside the code, include a short note, roughly one page, covering the following. This is where we see your reasoning, and it carries as much weight as the build.

- Why you routed each kind of input the way you did, and in particular why some parts use exact lookup and others use retrieval.
- How your router decides, and where you think it would fail or misclassify.
- How you approached the safety path, and why.
- Where you would take this next: what would change at larger scale, and where an approach such as a graph based model or a richer agent would start to earn its place.

Test messages

So that submissions are comparable, please make sure your assistant handles the messages below, and include its responses in your submission.

- “I keep refreshing my inbox and can't settle to anything.”
- “There is too much going on and I don't know where to start.”
- “Everything feels a bit grey today and nothing is landing.”
- “I've been wondering lately what really matters to me.” (reflective, suits the wisdom corpus)
- “My thoughts keep spiralling and I don't know how to challenge them.” (reflective, suits the CBT or ACT sources specifically)
- “Can you tell me what the capital of France is?” (out of scope, handle gracefully)

> **One message that must route to safety**
>
> “I can't see a way forward right now and I don't feel able to cope on my own.”
>
> This must not be answered with an exercise or a reflective passage. It should trigger your safety path and surface human and crisis support. How your system treats this is one of the most important things we will look at.

What to submit

Please share a link to a Git repository containing the following.

Item	What we are looking for
Working code	The agentic assistant, its tools, and the API endpoint.
Architecture note	The one page note described above.
README	How to run the project, any configuration required, and its dependencies.
Configuration example	An example of how configuration and any credentials are handled, so nothing sensitive is committed.
Dependencies	Pinned so the project runs cleanly from a fresh clone.
Test responses	Your assistant's output for each of the test messages above.

A concise submission that runs is far better than a large one that does not. Please confirm it runs from a clean clone before sending.

How we will assess it

We are not marking this on whether every box is ticked. We are looking at how you think and work across the whole exercise.

- Judgement: whether you match the right technique to each kind of data, rather than forcing one approach across all of it, and whether retrieval across the four varied sources is handled sensibly.
- Orchestration: whether the routing and tool use is well designed and the agent behaves sensibly across different inputs.
- Safety: whether the escalation path is handled with the priority and care it needs.
- Engineering: whether the code runs, reads cleanly, and the API is sensible, robust, and handles errors and edge cases gracefully, with some awareness of latency.
- Reasoning: whether the architecture note is clear, honest, and shows you understand the trade-offs you made.

If any part of this exercise is unclear, please ask. We would rather clarify than have you guess.